21EI43

USN | | | | | | | | | | |

RV COLLEGE OF ENGINEERING®

Autonomous Institution affiliated to VTU

IV Semester B. E. Examinations

ELECTRONICS AND INSTRUMENTATION ENGINEERING

Model Question Paper

21EI43 – Microcontroller & Programming

Time: 03 Hours **Maximum Marks: 100**

Instructions to candidates:

1. Answer all questions from Part A. Part A questions should be answered in first three pages of the answer book only.
2. Answer FIVE full questions from Part B. In Part B question number 2 is compulsory. Answer any one full question from 3 to 10.

PART-A

1	1.1	There are no 32-bit equivalent Thumb instructions.	1
	1.2	ARM: BL and BX instructions Instructions provide support for subroutine entry and exit.	2
	1.3	Each instruction in Cortex-M machines is encoded into 2-byte (16-bit)bytes word.	1
	1.4	How many bits data bus is used in Cortex?	1
	1.5	Which of the following instructions are accessing the memory locations? i) Store ii) MOVE iii) Load iv) arithmetic v) logical	2
	1.6	How many instructions pipelining is used in Cortex-M? 3-stage	1
	1.7	Identify the instructions which are called Program Status Register transfer instructions. MSR (Move to Status Register) MRS (Move from Status Register)	2
	1.8	Write an assembly instruction would you use to load 4 words starting from the memory location 0x80000000 into the registers r0-r3? (Assume r9 contains the base address 0x80000000). MOV r9, #0x80000000 ; Set r9 to the base address (0x80000000) LDR r0, [r9] ; Load the first word at memory location pointed by r9 into r0 ADD r9, r9, #4 ; Increment the base address by 4 bytes LDR r1, [r9] ; Load the second word into r1 ADD r9, r9, #4 ; Increment the base address by 4 bytes LDR r2, [r9] ; Load the third word into r2 ADD r9, r9, #4 ; Increment the base address by 4 bytes LDR r3, [r9] ; Load the fourth word into r3 In this code:	2
	1.9	When building code for different cortex states, compiler tool decides for each function call whether to use a BL or BLX instruction. (The linker is responsible for linking together object files and resolving symbols but doesn't typically make decisions about which specific instruction to use for function calls.)	1

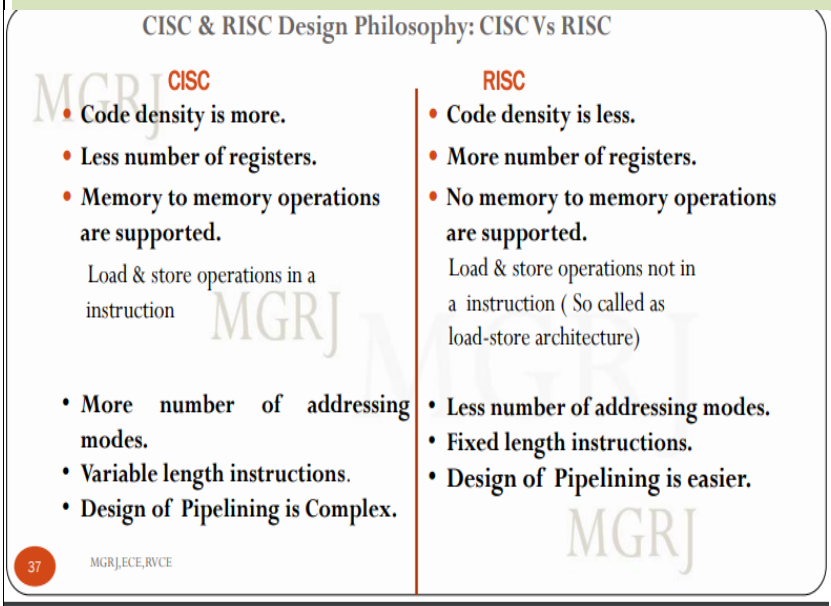
	1.10	ADD r3, r2, r1, LSL #3; instruction performs r3 is the destination register where the result will be stored. r2 and r1 are the source registers containing the values to be added. LSL #3 indicates that the value in r1 should be left-shifted by 3 bits before the addition._____.	1
	1.11	While CPU is executing a program, an interrupt exists then it stops_the normal sequence of execution of instructions (The CPU temporarily stops executing the current program, saves its context, and transfers control to an interrupt service routine (ISR) or handler. Once the ISR is executed, the CPU can return to the original program and continue its normal execution)	1
	1.12	In the branch instructions of Cortex, what does the mnemonic BVC imply? Branch if Overflow Clear. (" It's a conditional branch instruction that causes the program execution to branch to a new location in memory if the Overflow Flag (V) is clear. If the Overflow Flag is set (i.e., there has been an overflow condition in a previous instruction), the branch will not be taken, and execution continues with the next instruction in sequence. This instruction is often used in conditional branching based on arithmetic operations.)	1
	1.13	In LCD, which function is executed by '0x05' hex command? Shift display right	1
	1.14	In DC motor interfacing, which field/s is/are generated by forcing current through the coil for spinning of the motor? Magnetic field	1
	1.15	In DC motor interfacing, which modulation controls the duty cycle of square wave provided at the output by generating variation in the average DC voltage? Pulse Width Modulation	1

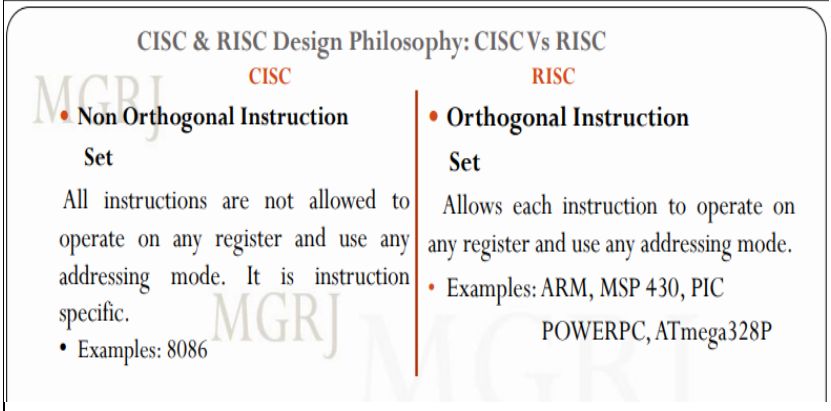
	1.16	While executing the main program, if two or more interrupts occur, then the sequence of appearance of interrupts is called nested interrupt and interrupt within interrupt	1

PART-B

2	a	Describe the evolution of ARM processor families. ARM1 (Acorn RISC Machine): The ARM architecture was initially developed by Acorn Computers in the 1980s. The ARM1 was the first ARM processor, released in 1985. ARM2 and ARM3: The ARM2 and ARM3 processors followed with improved performance and features. These early ARM processors were primarily used in Acorn's Archimedes computers. ARM6 and ARM7: The ARM6 and ARM7 processor families brought increased performance and were used in a wide range of embedded and portable devices. ARM7 introduced the Thumb instruction set, aimed at reducing code size for embedded systems. ARM9 and ARM10: The ARM9 and ARM10 families continued to improve performance and added features like floating-point support. These processors found use in various embedded systems, mobile devices, and networking equipment. ARM11: The ARM11 series provided a significant boost in processing power and multimedia capabilities. ARM11 processors were used in early smartphones and multimedia devices. Cortex-A Series: The Cortex-A series marked a major shift in ARM architecture, providing a range of high-performance processors suitable for use in smartphones, tablets, and other computing devices. Cortex-A8, A9, A15, and subsequent models offered increased performance, power efficiency, and support for features like multicore processing. Cortex-R Series: The Cortex-R series is designed for real-time and embedded applications, such as automotive systems and hard disk controllers. Cortex-R processors prioritize real-time performance and predictability. Cortex-M Series: The Cortex-M series is designed for microcontroller and embedded systems with a focus on energy efficiency and low cost. Cortex-M0, M3, M4, and M7 processors are widely used in various embedded applications. □ The Cortex-M3 and Cortex-M4 processors are based onARMv7-M architecture. □ Both are high-performance processors that are designed for microcontrollers. □ Cortex-M4 processor can also carry out some of the digital signal processing	6

		applications that traditionally have been carried out by a separate Digital Signal Processor. □ The Cortex-M0, Cortex-M0+, and the Cortex-M1 processors are based on ARMv6-M, which has a smaller instruction set.	
	b	Discuss the major design rules of RISC philosophy and also discuss CISC vs RISC Major design rules of RISC philosophy include: Simplified Instruction Set: RISC architectures use a reduced and highly optimized set of instructions. Each instruction typically performs a simple operation, which makes them fast to execute. Load and Store Architecture: RISC architectures typically separate load and store instructions for data memory access. This reduces complexity and allows more flexibility in instruction scheduling. Single-Cycle Execution: Instructions are executed in a single clock cycle, making the processor faster and more predictable. Hardwired Control Unit: RISC processors often use hardwired control units instead of microcode, which improves speed. Pipeline Execution: RISC processors are designed to execute instructions in a pipeline, which allows for concurrent execution of multiple instructions, improving throughput.	6





	c	Highlight the features of STM32F407 Processor. The STM32F407 processor is a high-performance 32-bit microcontroller based on the Arm Cortex-M4 core. It features a number of features that make it ideal for a wide range of embedded applications, including: High performance: The STM32F407 can operate at up to 168 MHz, making it one of the fastest Cortex-M4 processors available. Low power consumption: The STM32F407 features a number of power-saving features, such as dynamic voltage scaling and power gating, which make it ideal for battery-powered applications. Rich peripheral set: The STM32F407 includes a wide range of peripherals, including Ethernet, USB, CAN, and I2C. This makes it easy to connect the processor to other devices and sensors. Flexible memory architecture: The STM32F407 supports up to 1 Mbyte of Flash memory and 192 Kbytes of SRAM. This gives developers the flexibility to choose the right memory configuration for their application.	4

3	a	What are data processing instructions in STM32F407 processor? Define any ten data processing instructions and illustrate with an example for each. Data processing instructions in the STM32F407 processor are instructions that perform operations on data in registers. These instructions can be used to perform a variety of tasks, such as arithmetic operations, logical operations, and bitwise operations.	6

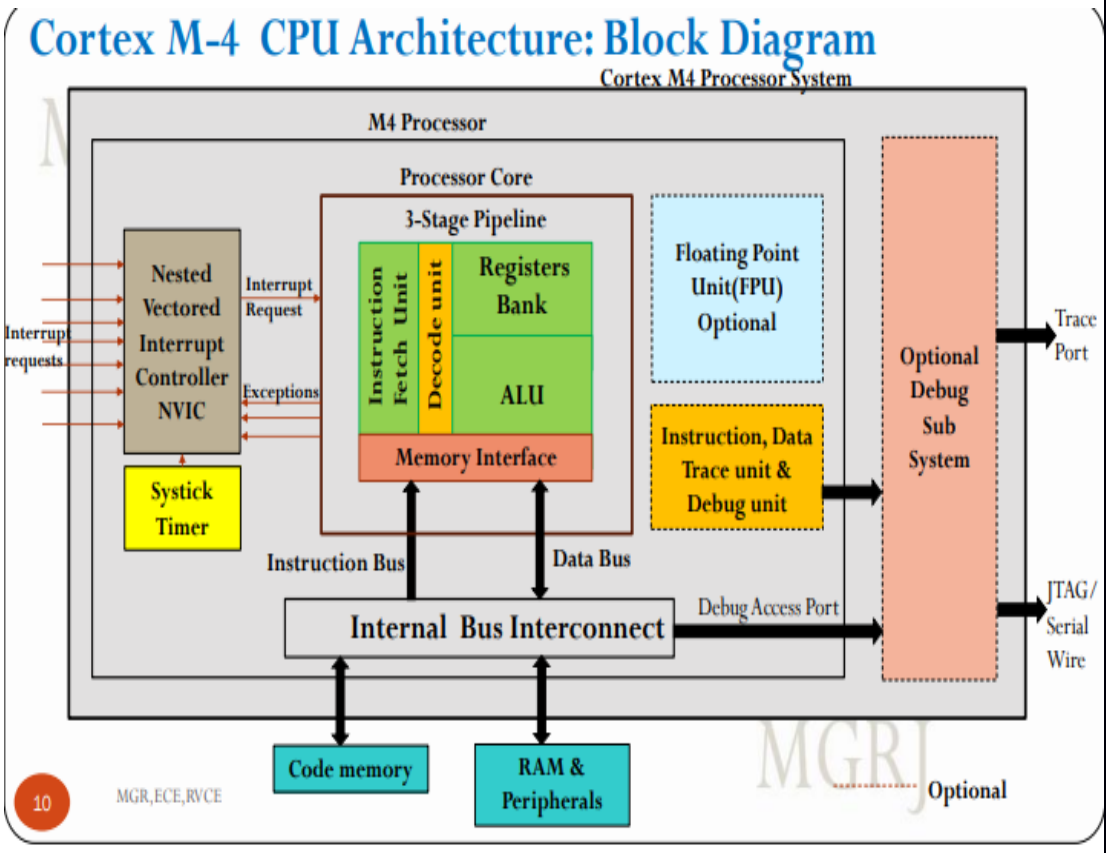
Instruction	Description	Example
MOV	Moves a value fr	MOV R0, R1 ; Move the value of register R1 to register R0.
ADD	Adds two values	ADD R0, R1, R2 ; Add the values of registers R1 and R2 and store the result in register R0.
SUB	Subtracts two va	SUB R0, R1, R2 ; Subtract the value of register R2 from the value of register R1 and store the result in reg
MUL	Multiplies two va	MUL R0, R1, R2 ; Multiply the values of registers R1 and R2 and store the result in register R0.
DIV	Divides two valu	DIV R0, R1, R2 ; Divide the value of register R1 by the value of register R2 and store the result in register
AND	Performs a logic	AND R0, R1, R2 ; Perform a logical AND operation on the values of registers R1 and R2 and store the res
OR	Performs a logic	OR R0, R1, R2 ; Perform a logical OR operation on the values of registers R1 and R2 and store the result
XOR	Performs a logic	XOR R0, R1, R2 ; Perform a logical XOR operation on the values of registers R1 and R2 and store the res
NOT	Performs a logic	NOT R0, R1 ; Perform a logical NOT operation on the value of register R1 and store the result in register
ASR	Performs an aritl	ASR R0, R1, #2 ; Perform an arithmetic shift right on the value of register R1 by 2 bits and store the result
LSL	Performs a logic	LSL R0, R1, #2 ; Perform a logical shift left on the value of register R1 by 2 bits and store the result in reg

	b	Illustrate the multiple memory loading and store instructions with an example for each. Multiple Memory Loading (LDM): The LDM (Load Multiple) instruction is used to load multiple registers with values from consecutive memory locations. This is useful when, for example, you want to load an array of data elements into registers. Example:LDMIA R0, {R1, R2, R3, R4} In this example: LDMIA loads the registers specified in the list (R1, R2, R3, R4) with values from memory. R0 is the base address from which the values are loaded. Multiple Memory Storing (STM): The STM (Store Multiple) instruction is used to store the values of multiple registers into consecutive memory locations. This is useful when you want to store data from multiple registers into an array in memory. Example:STMIA R0, {R1, R2, R3, R4} In this example: STMIA stores the values from registers R1, R2, R3, and R4 into memory. R0 is the base address where the values are stored. These examples demonstrate how the LDM and STM instructions are used to efficiently load and store multiple values between registers and memory in ARM assembly.	6
	c	Write an ARM Cortex-M Assembly Language Program to transfer the 10, 8-bits block of data from block of memory locations **X to Y.** LDR R0, =X ; Load the address of the source block into register R0 LDR R1, =Y ; Load the address of the destination block into register R1 MOV R2, #10 ; Loop over the 10 blocks of data LOOP: LDRB R3, [R0] ; Load the current byte from the source block into register R3 STRB R3, [R1] ; Store the current byte to the destination block ADD R0, R0, #1; Increment the addresses of the source and destination blocks ADD R1, R1, #1 SUB R2, R2, #1; Decrement the loop counter CMP R2, #0 BNE LOOP; Continue looping until all 10 blocks of data have been transferred MOV R0, #0; Exit the program BX LR	4

		OR	
4	a	Write an ARM Cortex-M Assembly language program to find the number of bits set in a 32-bit data, given by the user at 0x40000000 memory location. Store the result in 0x40000050, memory location.	8

```
; Load the 32-bit data from memory location 0x40000000 into register R0
LDR R0, =0x40000000

; Initialize a counter variable to store the number of bits set
MOV R1, #0

; Loop over the 32 bits of the data
MOV R2, #31
LOOP:

; Test the current bit of the data
AND R3, R0, #1

; If the current bit is set, increment the counter
CMP R3, #1
BNE SKIP
ADD R1, R1, #1

SKIP:

; Shift the data to the right by one bit
LSR R0, R0, #1

; Decrement the loop counter
SUB R2, R2, #1

; Continue looping until all 32 bits of the data have been tested
CMP R2, #0
BNE LOOP

; Store the number of bits set in memory location 0x40000050
STR R1, =0x40000050

; Exit the program
MOV R0, #0
BX LR
```

	b	With a neat block diagram, discuss ARM Cortex-M core data flow model.	8



Cortex M-4 CPU Architecture:

- The Cortex-M4 processors contain the core of the processor, the Nested Vectored Interrupt Controller (NVIC), the SysTick timer, and optionally the floating point unit (for Cortex-M4).
- Apart from these, the processors also contain some internal bus systems and a set of components to support debug operations.
- The processor core consists of 3-stage pipeline with fetch unit, decode unit and execution unit shown as register bank and ALU with separate buses for instruction fetch and data access.
- NVIC support up 240 peripheral interrupts and exceptions 1 to 15 are classified as system exceptions, and exceptions 16 and above are for interrupts.
- NVIC does priority based servicing, takes care of masking, pending requests, vector location for different interrupts.
- SysTick times 24-bit timer designed mainly for generating regular OS interrupts. It can also be used by application code if OS is not in use.

Cortex M-4 CPU Architecture:

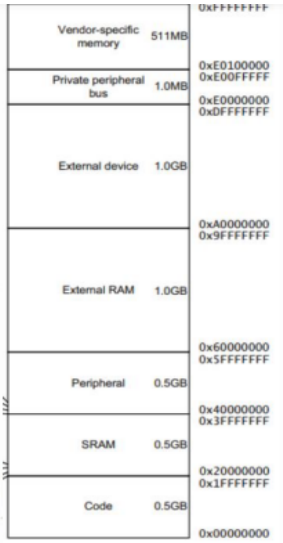
- Floating Point Unit(FPU) if present, perform operations in single precision on rational numbers. FPU is supported with separate registers and instruction set.
- The debug unit enables us to: stop program execution, examine and alter processor, examine and alter memory and input/output peripheral state.
- Instruction trace feature allows user to record of the instructions executed by the processor, enabling developers to analyze and debug the program flow at an instruction-level.
- Data trace allows the user to collect and analyze the data-related information during program execution especially memory read and write operations.
- A trace port, is a hardware interface used to capture and transmit trace data from a processor during program execution.
- JTAG (Joint Test Action Group) is a physical interface that allows for programming, debugging, and testing of these devices.

5	a	Describe the ARM Cortex Memory organization with neat diagram	6

Memory Organization

- The Cortex-M processors have 32-bit memory addressing and therefore have 4GB memory space.
- The memory space is unified, which means instructions and data share the same address space.
- Multiple bus interfaces to allow concurrent instructions and data accesses
- The memory map of cortex M4 CPUs is shown in figure.

Note: A memory map is a diagram or representation that illustrates the organization of a computer system's memory.



Memory Organization…

- As shown in the figure, the flash program memory on the microcontroller chip is mapped code region.
- The data memory is mapped to locations shown as SRAM in figure.
- The registers of different peripherals available in the microcontroller are mapped to peripheral region.
- External memory interface like SD card/SRAM/NAND Flash is mapped region external SRAM and external device.
- Private peripheral bus indicate the region of locations used for NVIC registers, FPU registers, Systick Timer registers, Trace and debug registers, etc.
- Vendor specific memory implementation varies with different vendors.

	b	Show how the processing time is reduced in Cortex-M compared to basic ARM instructions. ARM Cortex-M processors are designed to be more power-efficient and to provide improved processing time compared to basic ARM instructions. This is achieved through several architectural and design features. Here's a comparison of how Cortex-M processors reduce processing time compared to basic ARM instructions: Reduced Instruction Set: Cortex-M processors use a Reduced Instruction Set Computing (RISC) architecture with a smaller and more optimized instruction set compared to traditional ARM architectures. Fewer, simpler instructions allow for faster instruction fetch and execution. Single-Cycle Execution: Many Cortex-M instructions can be executed in a single clock cycle, providing faster execution times. Basic ARM instructions often require multiple clock cycles for execution. Pipeline Architecture: Cortex-M processors typically employ a pipeline architecture that allows multiple instructions to be in various stages of execution simultaneously. This parallelism reduces the overall processing time for a sequence of instructions. Harvard Architecture: Cortex-M processors use a Harvard architecture, which separates instruction and data memory. This separation allows for simultaneous fetching of instructions and data, further reducing processing time.	4
	c	Write a complete ARM Cortex-M **'C' program** for while loop using CubeMX tool for **Potentiometer interfacing** to perform **ADC**. Draw the interfacing diagram. Also write an algorithm for the same	6
		OR	
6	a	Write a complete ARM Cortex-M **'C' program** for while loop using CubeMX tool for Push button interfacing. Use blue LED**.** Also write an algorithm for the same.	8
	b	Write a complete ARM Cortex-M **'C' program** for while loop using CubeMX tool for **Potentiometer interfacing** to perform **DAC**. Draw the interfacing diagram. Also write an algorithm for the same	8

7	a	Write a **'C' program** using CubeMX tool to illustrate the USART transmission.	8
	b	Write a **'C' program** using CubeMX tool to demonstrate the SPI data transfer.	8
		OR	
8	a	Write an **'C' program** using CubeMX tool to configure the following: i) If PD.8 is set, then to rotate the stepper motor in clockwise rotation. else anticlockwise rotation. ii) PD.10 to control the speed of the motor.	8

	b	Configure the Seven segment display of ARM STM32F4XX board to display decimal digits and write a **'C' program** using CubeMX tool	8
9	a.	Illustrate the operation of timers available in Cortex Processor.	8

Timers in Cortex processors, specifically in the ARM Cortex-M series, play a crucial role in various embedded systems and real-time applications. These timers provide precise timing and can be used for various tasks, including generating interrupts, measuring time intervals, and controlling peripherals. Below, I'll illustrate the operation of two common types of timers in Cortex-M processors: SysTick and General-Purpose Timers.

1. SysTick Timer:

The SysTick timer is a simple and widely used timer in Cortex-M processors, especially in the context of real-time operating systems. It's a 24-bit down-counter, and its primary purpose is to provide a system timekeeping mechanism and generate periodic system timer interrupts.

Here's how the SysTick timer operates:

1. **Initialization**:
   - You configure the SysTick timer with a reload value. This value determines the time interval between SysTick timer interrupts.
   - You enable the SysTick timer and its interrupt.

2. **Counting**:
   - The timer counts down from the reload value to zero.
   - The timer decrements on each clock cycle.

3. **Interrupt Generation**:
   - When the timer reaches zero, it generates a SysTick interrupt.
   - This interrupt can be used for various timing and scheduling tasks in real-time systems.

4. **Reload**:
   - After reaching zero, the timer automatically reloads with the configured reload value.
   - The counting continues.

Example code to set up and use the SysTick timer in a Cortex-M microcontroller:

```c
// Set the SysTick reload value for a 1 ms interrupt
SysTick->LOAD = SystemCoreClock / 1000 - 1;
SysTick->VAL = 0;  // Clear the current value
SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
SysTick_CTRL_TICKINT_Msk | SysTick_CTRL_ENABLE_Msk;
```

2. General-Purpose Timers (TIM):

Cortex-M microcontrollers often include one or more general-purpose timers (e.g., TIM2, TIM3, etc.). These timers are versatile and can be configured for various tasks, including time measurement, pulse width modulation (PWM) generation, and

		event counting. Here's a simplified illustration of how a general-purpose timer operates: 1. **Initialization**: - Configure the timer's prescaler, which determines the timebase (time between each timer tick). - Set the auto-reload value, which determines when the timer should reset or generate an update event. - Define the mode of operation (count up, count down, etc.). 2. **Counting**: - The timer counts up or down based on the selected mode and the clock input. 3. **Interrupt Generation**: - When the timer reaches the auto-reload value, it generates an update event interrupt. - This interrupt can be used for precise timing tasks and time measurement. 4. **PWM Generation (Optional)**: - General-purpose timers can be configured to generate PWM signals on their output channels. The duty cycle and frequency of the PWM signal are determined by the timer's settings. Example code to set up a general-purpose timer (TIM2) for basic timing: ```c // Initialize TIM2 TIM2->PSC = 999; // Set the prescaler (for a 1 kHz clock) TIM2->ARR = 999; // Set the auto-reload value (for a 1 Hz interrupt) TIM2->DIER = TIM_DIER_UIE; // Enable update event interrupt TIM2->CR1 = TIM_CR1_CEN; // Start the timer ``` These are simplified illustrations of the operation of SysTick and general-purpose timers in Cortex-M processors.	
	b.	Write a **'C' program** using CubeMX tool to demonstrate the operation of pulse width modulation.	8

		OR	
10	a.	List the various interrupt handling schemes. With an example clearly highlight the mechanism of nested vector interrupt handler. Interrupt handling schemes are strategies and mechanisms used to manage and prioritize multiple interrupts in microcontroller and processor systems. One common scheme used in ARM Cortex-M processors is the Nested Vector Interrupt Controller (NVIC). Here are various interrupt handling schemes, with an example highlighting the mechanism of a Nested Vector Interrupt Handler: **Various Interrupt Handling Schemes**: 1. **Simple Interrupt Handling**: - In this scheme, only one interrupt can be serviced at a time. - When an interrupt occurs, the processor immediately branches to the corresponding interrupt service routine (ISR). - The ISR executes, and once it's done, the processor returns to the main program. - Any other pending interrupts must wait until the current ISR is complete. 2. **Priority Interrupt Handling**: - In this scheme, interrupts are assigned different priority levels. - When multiple interrupts occur, the processor services the interrupt with the highest priority. - Lower-priority interrupts are deferred until the higher-priority interrupt is completed, allowing for better control in a multi-priority environment. 3. **Vector Interrupt Handling**: - In this scheme, a vector table is used to store the addresses of various interrupt service routines (ISRs). - When an interrupt occurs, the processor looks up the corresponding address in the vector table and branches to the appropriate ISR. - This allows for a more organized and structured handling of interrupts. 4. **Nested Vector Interrupt Controller (NVIC)**: - NVIC is a sophisticated scheme used in ARM Cortex-M processors. - It combines both priority and vector interrupt handling, offering a flexible approach to managing multiple interrupts. - NVIC allows for the handling of multiple interrupts in a nested manner, with different priority levels. - High-priority interrupts can preempt lower-priority ones, enabling efficient and responsive interrupt management. **Example of Nested Vector Interrupt Handling (NVIC)**: In an ARM Cortex-M microcontroller, such as the STM32 series, the NVIC is responsible for managing and prioritizing interrupts. Let's consider an example with two interrupts: Button Press (higher priority) and Timer (lower priority). ```c #include "stm32f4xx.h" void EXTI0_IRQHandler(void) { // Handle Button Press Interrupt	8

		} void TIM2_IRQHandler(void) { // Handle Timer Interrupt } int main(void) { // NVIC configuration NVIC_SetPriority(EXTI0_IRQn, 0); // Button Press has the highest priority NVIC_SetPriority(TIM2_IRQn, 1); // Timer has lower priority NVIC_EnableIRQ(EXTI0_IRQn); // Enable Button Press interrupt NVIC_EnableIRQ(TIM2_IRQn); // Enable Timer interrupt while (1) { // Main program loop } return 0; } ``` In this example, we have two interrupts: Button Press (EXTI0) and Timer (TIM2). The NVIC is configured with different priority levels. When multiple interrupts occur, the NVIC ensures that the higher-priority Button Press interrupt preempts the lower-priority Timer interrupt. This mechanism allows efficient and structured handling of different interrupt types with varying priorities.	
	b.	What are the different exceptions the programmer should take care while writing an optimized code? 1. **Exception Handling Overhead**: - Exception handling introduces overhead. Minimize the use of exceptions or use them judiciously to avoid performance penalties. 2. **Interrupt Latency**: - Be mindful of interrupt latency, the time it takes for the processor to respond to an interrupt. Minimize interrupt latency to meet real-time requirements. 3. **Memory Alignment**: - ARM processors may require certain data structures to be aligned in memory for optimal performance. Ensure proper data alignment, especially for data accessed by SIMD (Single Instruction, Multiple Data) instructions. 4. **Pipeline Stall and Branches**: - Reduce pipeline stalls caused by mispredicted branches. Avoid conditional branches in performance-critical loops when possible. 5. **Data Caching**: - Understand the impact of data caching on performance. Optimize memory access patterns to take advantage of caching mechanisms. 6. **Instruction Caches**: - Consider code alignment and instruction cache optimization to minimize	8

instruction cache misses, which can slow down program execution.

7. **Loop Unrolling**:
   - Use loop unrolling when it improves performance. Unrolling loops can reduce overhead and make better use of available registers.

8. **Function Inlining**:
   - Consider inlining small, frequently called functions to reduce the overhead of function calls.

9. **Register Usage**:
   - Efficiently use registers, as ARM processors have a limited number of registers available. Avoid spilling data to memory when possible.

10. **Thumb vs. ARM Mode**:
    - In the ARM Cortex-M series, consider using the Thumb instruction set to reduce code size. This can be especially helpful in memory-constrained systems.

11. **Floating-Point Optimization**:
    - Optimize floating-point operations when using ARM processors with hardware floating-point units (FPU). Take advantage of SIMD instructions for parallel processing when dealing with floating-point data.

12. **Compiler Optimization Settings**:
    - Leverage compiler optimization flags to automatically apply various code optimizations. Be aware of compiler-specific directives and attributes.

13. **Pipeline-Friendly Code**:
    - Write code that minimizes data dependencies to keep the pipeline flowing smoothly. Understand the concept of instruction pipelines in modern ARM processors.

14. **Data Transfer and Endianness**:
    - Be aware of data transfer overhead, especially in situations where data needs to be byte-swapped due to endianness differences between systems.

15. **Conditional Code Execution**:
    - Use conditional execution and predication to execute code conditionally, reducing the need for branching.

16. **Compiler Barriers**:
    - Use compiler memory barriers and synchronization mechanisms to ensure that data consistency is maintained in a multi-threaded or multi-core environment.

17. **Cache and Memory Hierarchy**:
    - Understand the memory hierarchy, including L1 and L2 caches, and optimize data access patterns accordingly.

18. **Resource Utilization**:
    - Efficiently use on-chip resources, such as memory, cache, and coprocessors, to maximize performance and minimize resource contention.